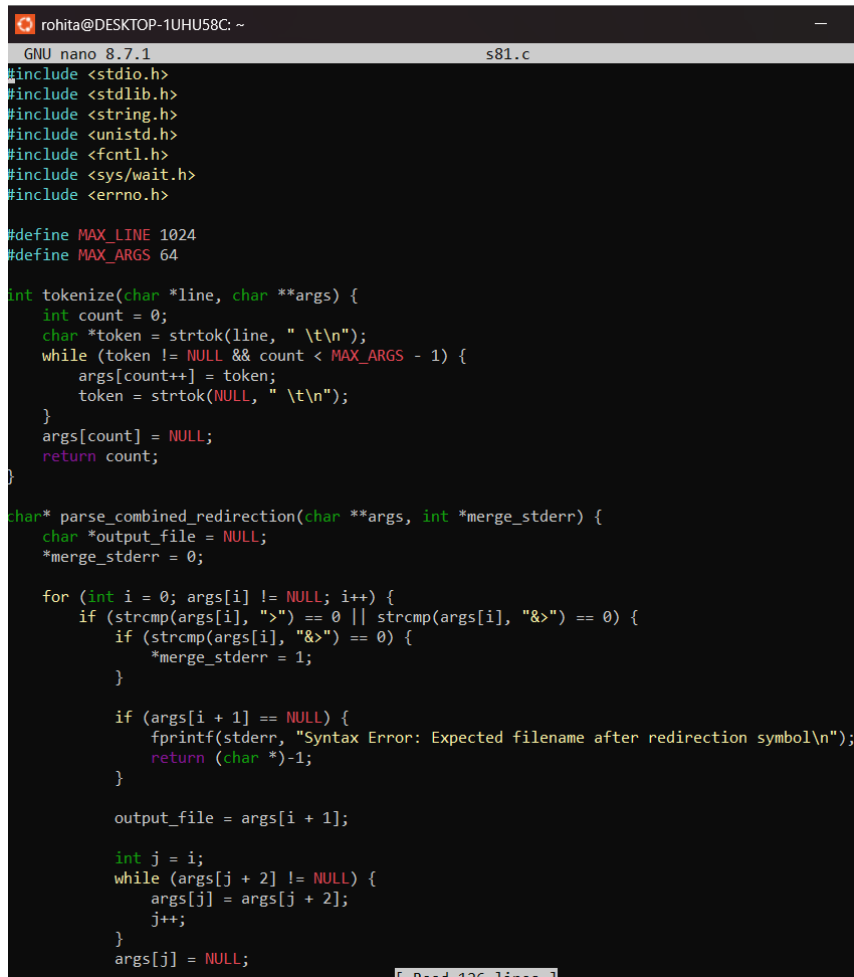


Skill 8 -OS-2520090235

SESSION 15

Task 1: To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

Code:



```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s81.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_LINE 1024
#define MAX_ARGS 64

int tokenize(char *line, char **args) {
    int count = 0;
    char *token = strtok(line, " \\t\\n");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    args[count] = NULL;
    return count;
}

char* parse_combined_redirection(char **args, int *merge_stderr) {
    char *output_file = NULL;
    *merge_stderr = 0;

    for (int i = 0; args[i] != NULL; i++) {
        if (strcmp(args[i], ">") == 0 || strcmp(args[i], "&>") == 0) {
            if (strcmp(args[i], "&>") == 0) {
                *merge_stderr = 1;
            }

            if (args[i + 1] == NULL) {
                fprintf(stderr, "Syntax Error: Expected filename after redirection symbol\\n");
                return (char *)-1;
            }

            output_file = args[i + 1];

            int j = i;
            while (args[j + 2] != NULL) {
                args[j] = args[j + 2];
                j++;
            }
            args[j] = NULL;
        }
    }
}
```

```

        args[j] = NULL;
        i--;
    } else if (strcmp(args[i], ">&1") == 0) {
        *merge_stderr = 1;

        int j = i;
        while (args[j + 1] != NULL) {
            args[j] = args[j + 1];
            j++;
        }
        args[j] = NULL;
        i--;
    }
}
return output_file;
}

int main(void) {
    char line[MAX_LINE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        int num_args = tokenize(line, args);
        if (num_args == 0) continue;

        if (strcmp(args[0], "exit") == 0) break;

        int merge_stderr = 0;
        char *output_file = parse_combined_redirection(args, &merge_stderr);
        if (output_file == (char *)-1) continue;

        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            continue;

```

```

        continue;
    }

    if (pid == 0) {
        if (output_file != NULL) {
            int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

            if (fd < 0) {
                fprintf(stderr, "myshell: %s: %s\n", output_file, strerror(errno));
                exit(EXIT_FAILURE);
            }

            if (dup2(fd, STDOUT_FILENO) < 0) {
                perror("dup2 failed for stdout");
                close(fd);
                exit(EXIT_FAILURE);
            }

            if (merge_stderr) {
                if (dup2(STDOUT_FILENO, STDERR_FILENO) < 0) {
                    perror("dup2 failed for stderr");
                    close(fd);
                    exit(EXIT_FAILURE);
                }
            }

            close(fd);
        }

        execvp(args[0], args);
        fprintf(stderr, "myshell: command not found: %s\n", args[0]);
        exit(EXIT_FAILURE);
    } else {
        int status;
        waitpid(pid, &status, 0);
    }
}

return 0;
}

```

Output:

```
rohita@DESKTOP-1UHU58C:~$ ./a.out
myshell> ls main.c non_existent.txt > combined.log 2>&1
myshell> cat combined.log
ls: cannot access 'main.c': No such file or directory
ls: cannot access 'non_existent.txt': No such file or directory
myshell> ls main.c non_existent.txt &> combined2.log
myshell> cat combined2.log
ls: cannot access 'main.c': No such file or directory
ls: cannot access 'non_existent.txt': No such file or directory
myshell> exit
rohita@DESKTOP-1UHU58C:~$
```

Observation: dup2() sends both **stdout** and **stderr** to the same file. Doing this in the **child process** keeps the parent shell unchanged.

Task 2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

Code:

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s82.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_CMD 20
#define MAX_LEN 500

typedef struct {
    char command[MAX_LEN];
    char input[MAX_LEN];
    char output[MAX_LEN];
    int append;
} Command;

void trim(char *str) {
    int start = 0, end = strlen(str) - 1;

    while (isspace(str[start]))
        start++;

    while (end >= start && isspace(str[end]))
        end--;

    memmove(str, str + start, end - start + 1);
    str[end - start + 1] = '\0';
}

int validate(char *cmd) {
    int len = strlen(cmd);

    if (len == 0)
        return 0;

    if (cmd[0] == '|' || cmd[len - 1] == '|') {
        printf("Error: Pipe cannot be at beginning or end.\n");
        return 0;
    }

    for (int i = 0; i < len; i++) {
        if (cmd[i] == '|') {
            int j = i + 1;

            while (j < len && isspace(cmd[j]))
```

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
if (in != NULL) {
    *in = '\0';

    char *filename = in + 1;
    trim(filename);

    strcpy(c->input, filename);
}

/* Output redirection */
char *out = strstr(temp, ">>");

if (out != NULL) {
    *out = '\0';

    char *filename = out + 2;
    trim(filename);

    strcpy(c->output, filename);
    c->append = 1;
}
else {
    out = strchr(temp, '>');

    if (out != NULL) {
        *out = '\0';

        char *filename = out + 1;
        trim(filename);

        strcpy(c->output, filename);
        c->append = 0;
    }
}

trim(temp);
strcpy(c->command, temp);

int main() {

    char input[MAX_LEN];

    Command commands[MAX_CMD];
    int count = 0;

    while (j < len && isspace(cmd[j]))
        j++;

    if (j < len && cmd[j] == '|') {
        printf("Error: Consecutive pipes are not allowed.\n");
        return 0;
    }

    if (cmd[i] == '<' || cmd[i] == '>') {
        int j = i + 1;

        while (j < len && isspace(cmd[j]))
            j++;

        if (j >= len || cmd[j] == '|' ||
            cmd[j] == '<' || cmd[j] == '>') {
            printf("Error: Missing filename after redirection.\n");
            return 0;
        }
    }

    return 1;
}

void parseCommand(char *str, Command *c) {

    char temp[MAX_LEN];
    strcpy(temp, str);

    c->input[0] = '\0';
    c->output[0] = '\0';
    c->append = 0;

    /* Input redirection */
    char *in = strchr(temp, '<');

    if (in != NULL) {
        *in = '\0';

        char *filename = in + 1;

```

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s82.c

printf("=====\n");
printf("    COMPLEX COMMAND PARSER\n");
printf("=====\n");

printf("\nEnter command:\n$ ");

fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';

/* Validate syntax */
if (!validate(input)) {
    printf("\nSyntax validation failed.\n");
    return 1;
}

/* Split using pipe */
char *part = strtok(input, "|");

while (part != NULL && count < MAX_CMD) {

    trim(part);

    parseCommand(part, &commands[count]);

    count++;

    part = strtok(NULL, "|");
}

/* Display parsed commands */
printf("\n-----\n");
printf("PARSED COMMANDS\n");
printf("-----\n");

for (int i = 0; i < count; i++) {

    printf("\nCommand %d:\n", i + 1);

    printf("  Command : %s\n",
           commands[i].command);

    if (strlen(commands[i].input) > 0)
        printf("  Input   : %s\n",
               commands[i].input);
}

```

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s82.c

    if (strlen(commands[i].input) > 0)
        printf("  Input   : %s\n",
               commands[i].input);
    else
        printf("  Input   : STDIN\n");

    if (strlen(commands[i].output) > 0) {
        printf("  Output  : %s\n",
               commands[i].output);

        if (commands[i].append)
            printf("  Mode    : APPEND (>>)\n");
        else
            printf("  Mode    : OVERWRITE (>)\n");
    }
    else {
        printf("  Output  : STDOUT\n");
    }
}

/* Generate execution plan */
printf("\n-----\n");
printf("EXECUTION PLAN\n");
printf("-----\n");

for (int i = 0; i < count; i++) {

    printf("\nStep %d: Execute \"%s\"",
           i + 1,
           commands[i].command);

    if (i < count - 1)
        printf(" | create pipe to next command");

    if (strlen(commands[i].input) > 0)
        printf(" | read from \"%s\"",
               commands[i].input);

    if (strlen(commands[i].output) > 0) {

        if (commands[i].append)
            printf(" | append output to \"%s\"",
                   commands[i].output);
        else

```

```

        commands[i].output);
    else
        printf(" | redirect output to \"%s\\n\"",
               commands[i].output);
    }
    printf("\\n");
}

printf("\\n-----\\n");
printf("PARSING SUCCESSFUL\\n");
printf("-----\\n");

return 0;
}

```

Output:

```

=====
      COMPLEX COMMAND PARSER
=====

Enter command:
$ cat < input.txt | grep hello > output.txt

-----
PARSED COMMANDS
-----

Command 1:
  Command : cat
  Input   : input.txt
  Output  : STDOUT

Command 2:
  Command : grep hello
  Input   : STDIN
  Output  : output.txt
  Mode    : OVERWRITE (>)

-----
EXECUTION PLAN
-----

Step 1: Execute "cat" | create pipe to next command | read from "input.txt"
Step 2: Execute "grep hello" | redirect output to "output.txt"

-----
PARSING SUCCESSFUL
-----

```

Observation: The program successfully parsed complex commands containing pipes and input/output redirections. It validated the command syntax and generated the correct execution plan showing the order of command execution.

SESSION 16

Task 1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

Code:

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20
#define MAX_CMD 100

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_CMD];
    int running;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, char *cmd) {
    if (job_count >= MAX_JOBS) {
        printf("Job limit reached!\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, cmd);
    jobs[job_count].running = 1;

    printf("[%d] %d running in background: %s\n",
           jobs[job_count].job_id, pid, cmd);

    job_count++;
}

void check_jobs() {
    int status;

    for (int i = 0; i < job_count; i++) {

        if (jobs[i].running) {

            pid_t result = waitpid(
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == 0) {
            printf("[%d] PID %d still running: %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
        else if (result == jobs[i].pid) {
            jobs[i].running = 0;

            printf("[%d] PID %d completed: %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
    }
}

void show_jobs() {
    printf("\n--- Background Jobs ---\n");

    for (int i = 0; i < job_count; i++) {

        printf("[%d] PID=%d %s [%s]\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].command,
            jobs[i].running ? "Running" : "Completed");
    }
}

int main() {

    char command[MAX_CMD];

    printf("=====\n");
    printf("      BACKGROUND JOB MANAGER\n");
    printf("=====\n");
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
printf("Commands:\n");
printf("  command &  -> Run in background\n");
printf("  jobs       -> Show background jobs\n");
printf("  check        -> Check job status\n");
printf("  exit         -> Exit program\n\n");

while (1) {

    check_jobs();

    printf("\n$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0) {
        break;
    }

    if (strcmp(command, "jobs") == 0) {
        show_jobs();
        continue;
    }

    if (strcmp(command, "check") == 0) {
        check_jobs();
        continue;
    }

    int len = strlen(command);

    /* Detect background operator & */
    if (len > 0 && command[len - 1] == '&') {

        command[len - 1] = '\0';

        /* Remove trailing spaces */
        while (len > 0 &&
            command[len - 1] == ' ') {
            command[len - 1] = '\0';
            len--;
        }
    }
}
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    continue;
}

if (pid == 0) {

    /* Background child process */
    execlp("sh", "sh", "-c", command, NULL);

    perror("exec failed");
    exit(1);
}

else {
    /* Parent does NOT wait */
    add_job(pid, command);

    printf("Parent process continues immediately.\n");
}

else {

    printf("Running foreground command: %s\n",
           command);

    pid_t pid = fork();

    if (pid == 0) {
        execlp("sh", "sh", "-c", command, NULL);
        perror("exec failed");
        exit(1);
    }

    else if (pid > 0) {
        waitpid(pid, NULL, 0);
        printf("Foreground command completed.\n");
    }

    else {
        perror("fork failed");
    }
}
```

```

    }

    else if (pid > 0) {
        waitpid(pid, NULL, 0);
        printf("Foreground command completed.\n");
    }

    else {
        perror("fork failed");
    }
}

printf("\nExiting Background Job Manager...\n");

return 0;
}

```

Output:

```

=====
      BACKGROUND JOB MANAGER
=====
Commands:
  command &  -> Run in background
  jobs       -> Show background jobs
  check      -> Check job status
  exit       -> Exit program

$ check

$ command &
[1] 8475 running in background: command
Parent process continues immediately.
[1] PID 8475 still running: command

$ jobs

--- Background Jobs ---
[1] PID=8475  command  [Running]
[1] PID 8475 completed: command

$ check

$ exit

```

Observation: The program successfully detects commands ending with &, launches them as non-blocking background processes using fork() and exec(), and immediately returns the prompt. It stores job information and monitors the execution status using waitpid() with WNOHANG.

Task 2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 s84.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <signal.h>  
#include <string.h>  
  
#define MAX_JOBS 10  
  
typedef enum {  
    RUNNING,  
    STOPPED,  
    COMPLETED  
} JobState;  
  
typedef struct {  
    int job_id;  
    pid_t pid;  
    pid_t pgid;  
    char command[100];  
    JobState state;  
} Job;  
  
Job jobs[MAX_JOBS];  
int job_count = 0;  
  
void add_job(pid_t pid, pid_t pgid, char *cmd) {  
    if (job_count >= MAX_JOBS) {  
        printf("Job table is full.\n");  
        return;  
    }  
  
    jobs[job_count].job_id = job_count + 1;  
    jobs[job_count].pid = pid;  
    jobs[job_count].pgid = pgid;  
    strcpy(jobs[job_count].command, cmd);  
    jobs[job_count].state = RUNNING;  
  
    job_count++;  
}  
  
void update_jobs() {  
    int status;  
  
    for (int i = 0; i < job_count; i++) {
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
        if (jobs[i].state == RUNNING) {

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result == jobs[i].pid) {
                jobs[i].state = COMPLETED;
            }
        }
    }

void remove_completed_jobs() {
    for (int i = 0; i < job_count; i++) {

        if (jobs[i].state == COMPLETED) {

            for (int j = i; j < job_count - 1; j++) {
                jobs[j] = jobs[j + 1];
            }

            job_count--;
            i--;
        }
    }
}

void display_jobs() {

    printf("\n===== JOB TABLE =====\n");

    if (job_count == 0) {
        printf("No active jobs.\n");
        return;
    }

    printf("ID\tPID\tPGID\tSTATE\t\tCOMMAND\n");

    for (int i = 0; i < job_count; i++) {

        char *state;
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
        if (jobs[i].state == RUNNING)
            state = "RUNNING";
        else if (jobs[i].state == STOPPED)
            state = "STOPPED";
        else
            state = "COMPLETED";

        printf("%d\t%d\t%d\t%s\t\t%s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].pgid,
                state,
                jobs[i].command);
    }
}

int main() {

    char command[100];

    printf("=====\n");
    printf("      JOB TABLE MANAGER\n");
    printf("=====\n");

    while (1) {

        update_jobs();

        printf("\n$ ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            break;
        }

        if (strcmp(command, "jobs") == 0) {
            display_jobs();
            continue;
        }

        if (strcmp(command, "remove") == 0) {
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
    remove_completed_jobs();
    printf("Completed jobs removed.\n");
    continue;
}

/* Background command */
int len = strlen(command);

if (len > 0 && command[len - 1] == '&') {
    command[len - 1] = '\\0';

    while (strlen(command) > 0 &&
           command[strlen(command) - 1] == ' ') {
        command[strlen(command) - 1] = '\\0';
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        continue;
    }

    if (pid == 0) {

        /* Create a new process group */
        setpgid(0, 0);

        execlp("sh", "sh", "-c", command, NULL);

        perror("exec");
        exit(1);
    }

    else {

        /* Parent stores job information */
        setpgid(pid, pid);

        add_job(pid, pid, command);

        printf("Job added successfully.\n");
        printf("PID   : %d\n", pid);
        printf("PGID  : %d\n", pid);
    }
}
```

```

        printf("Job added successfully.\n");
        printf("PID : %d\n", pid);
        printf("PGID : %d\n", pid);
    }
}

else {
    printf("Use '&' to create a background job.\n");
    printf("Example: sleep 10 &\n");
}

return 0;
}

```

Output:

```

rohita@DESKTOP-1UHU58C:~$ gcc s84.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
=====
      JOB TABLE MANAGER
=====

$ sleep 20 &
Job added successfully.
PID : 8555
PGID : 8555

$ jobs

===== JOB TABLE =====
ID      PID    PGID    STATE    COMMAND
1       8555    8555    RUNNING  sleep 20

$ jobs

===== JOB TABLE =====
ID      PID    PGID    STATE    COMMAND
1       8555    8555    RUNNING  sleep 20

$ remove
Completed jobs removed.

$ =

```

Observation: The program successfully maintains a job table containing **job ID, PID, process group ID, command, and state**. It updates job status, validates job records, and removes completed jobs from the table.

